

Sistem Operasi (INF-201)

Pertemuan 5: Sinkronisasi Proses & Concurrency

Tim Dosen Sistem Operasi

Program Studi Informatika – Universitas Bengkulu

Semester Ganjil 2026/2027

Capaian Pembelajaran (Sub-CPMK 5)

- Mahasiswa mampu memecahkan masalah Race Condition pada sistem multi-threading.
- Mahasiswa memahami mekanisme Mutex Locks dan Semaphores (Counting & Binary).
- Mahasiswa mampu menguji dan mendeteksi data race menggunakan ThreadSanitizer.

Race Condition & Critical Section Problem

Race Condition

Kondisi di mana beberapa thread/proses mengakses dan memodifikasi data bersama (*shared data*) secara *concurrent*, sehingga hasil akhir tergantung pada urutan eksekusi thread.

3 Syarat Solusi Critical Section:

- ① **Mutual Exclusion:** Hanya 1 thread yang boleh berada di Critical Section dalam satu waktu.
- ② **Progress:** Pemilihan thread berikutnya tidak boleh tertunda tanpa alasan.
- ③ **Bounded Waiting:** Ada batas berapa kali thread lain boleh masuk sebelum thread yang meminta diizinkan masuk.

Sinkronisasi POSIX Threads (pthread) & Mutex

```
#include <pthread.h>

int counter = 0;
pthread_mutex_t lock;

void* count_up(void* arg) {
    pthread_mutex_lock(&lock);
    counter++; // Critical Section
    pthread_mutex_unlock(&lock);
    return NULL;
}
```

Tugas Terstruktur 2: POSIX Thread Synchronizer

Praktikum 2 (Tugas Terstruktur - Bobot 6%)

Mengimplementasikan penyelesaian masalah *Producer-Consumer* atau *Dining Philosophers* menggunakan **POSIX Threads ('pthread')**, **Mutex**, dan **Semaphores**.

- Gunakan flag '-fsanitize=thread' pada 'gcc' saat kompilasi untuk memverifikasi bahwa program bebas dari *Data Race*.